Software processes

Saturday, October 11, 2025 5:13 PM

Software Processes

- A set of related activities that leads to the production of a software system.
- **Key point**: There is **no universal software engineering method** that fits all types of software systems.
- The choice of process depends on:
  - Type of software being developed
  - Customer requirements
  - Skills of the development team

Fundamental Software Engineering Activities

All software processes, regardless of type, include four main activities:

- **Software specification**: Understanding and defining what services are required.
- **Software development**: Converting design into an executable system.
- **Software validation**: Ensuring the system meets specifications and user expectations.
- **Software evolution**: Maintaining and adapting the system to meet changing customer needs.

Real-life example:

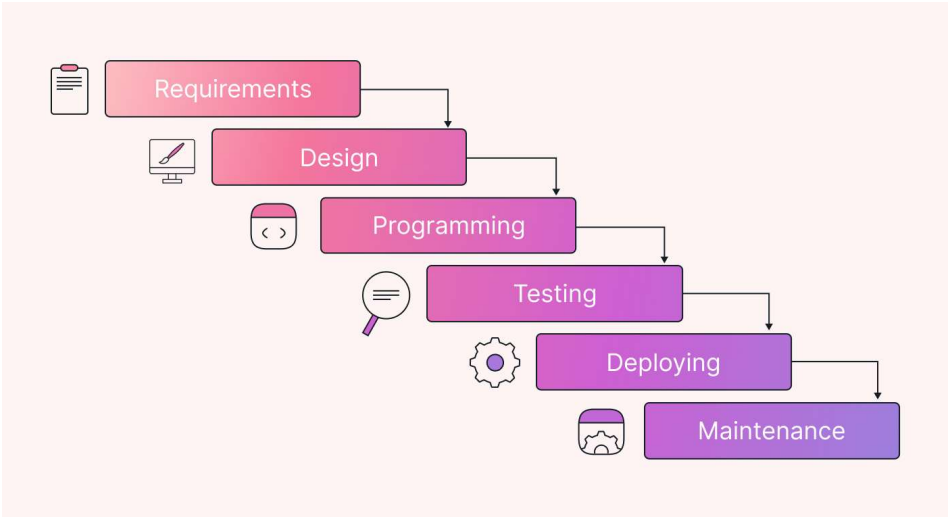
- Developing a **banking app**:
  - **Specification**: Determine features like fund transfer, balance check, bill payments.
  - **Development**: Writing the code for these features.
  - **Validation**: Testing if fund transfers are accurate and secure.
  - **Evolution**: Adding new features such as UPI payments or improving UI based on user feedback.

Software Process Models

- An abstract representation of a software process, showing the **order in which activities are carried out**.
- **Purpose**: Helps plan, manage, and execute software development effectively.

1) Waterfall Model

- **Approach**: Sequential, linear process where each phase is **completed before moving to the next**.
- **Characteristics**:
  - Phases are strictly sequential.
  - Well-documented and predefined.
  - Features expected in each phase are clearly defined.



Phases

1. **Requirements Gathering and Analysis**
   - Capture all possible requirements of the system.
   - Document them in a **requirement specification document**.
2. **System Design**
   - Prepare system design based on the requirements.
   - Identify architecture, data flow, and modules.
3. **Implementation**
   - Programmers develop the system in small **units**.
   - Units are tested individually.
4. **Integration and Testing**
   - Combine all units into a complete system.
   - Test the system for functionality and correctness.
5. **Deployment and Maintenance**
   - Deploy the system in the customer environment.
   - Perform maintenance for **bug fixes** and **enhancements**.

Merits

- Simple to implement.
- Requires minimal resources.
- Best for **simple, well-defined requirements** that are unlikely to change.
- Fixed start and end points make progress tracking easy.
- Easy to manage due to **rigid structure**.

Demerits

- No working software until late in the life cycle.
- Cannot accommodate **requirement changes** during development.
- Hard to go back to previous phases.
- Risk reduction is difficult since **testing occurs late**.

Real-life Example:

- Developing a **library management system** where the requirements are clear:
  - Track books, issue/return books, generate reports.
  - Since requirements are fixed, a Waterfall approach works well.

2) Incremental Development Model

In the incremental model, the system is built and delivered in **small increments**, with each increment providing part

of the system functionality. Customers identify which features are most important, and high-priority features are delivered first. Each increment can be tested and used by the customer, helping refine requirements for future increments.

Key Points:

- System functionality grows with each delivered increment.
- Customers can give feedback after each increment.
- Requirements clarification happens continuously.

Example (Real-life):

- Consider a **banking app**. Initially, the first increment may include only **viewing balance and transaction history**.
- The next increment may add **fund transfer**, and the following increment may add **bill payments and card management**.
- Each increment is integrated with the previous, improving the system gradually.

3) Integration and Configuration Model (Reuse-Oriented Development)

This model emphasizes **reusing existing software components** to build new systems, reducing development time and cost. Developers search for suitable components, adapt them, and integrate them with newly developed code.

Stages:

Example: Banking App

1. **Requirement Specification:** Capture all features needed, e.g., balance inquiry, fund transfer, bill payments, and mini-statements.
   Example: Customer wants a mobile app to transfer money and check account balance.
2. **Component Analysis:** Search for existing software modules that can provide these features.
   Example: Use an existing secure payment gateway module or transaction history module.
3. **Requirements Modification:** Adjust the requirements to align with what components can provide.
   Example: If the existing module supports only local transfers, update the app requirement to initially allow only local transfers.
4. **System Design with Reuse:** Design the app combining reused components and plan new components if needed.
   Example: Integrate payment module, balance inquiry module, and create a new module for bill payments.
5. **Development and Integration:** Combine all reused and newly developed modules into a complete app.
   Example: Build the app so the user can seamlessly check balance, transfer funds, and pay bills in one interface.
6. **System Validation:** Test the app to ensure it works correctly, securely, and meets customer expectations.
   Example: Verify that all transactions are accurate, secure, and that the app handles errors gracefully.

Coping with Change in Software Projects

Change is inevitable in large software projects because business requirements evolve due to external pressures, competition, or management priorities. Change increases development costs, as completed work may need to be redone.

Two main approaches to cope with change:

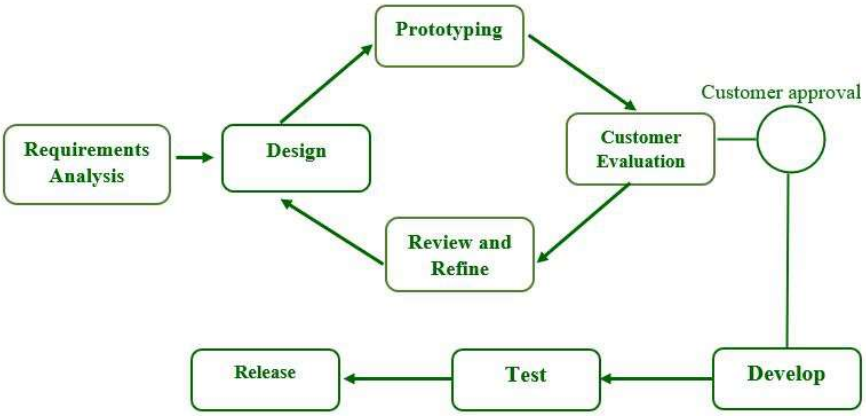
1) Prototyping

- Instead of detailed specifications upfront, developers create a **quick prototype** containing only part of the system.
- The prototype is **iteratively refined** until the final specifications are achieved.
- **Uses:**
  - **Requirements Engineering:** To elicit and validate system requirements.
  - **System Design:** To explore software solutions and develop user interfaces.

Example: For a **banking app**, a simple prototype may allow users to see how the login, balance check, and fund transfer screens will look. Feedback helps refine the actual system.

How it helps in coping with change:

- In large software projects, requirements often change due to business, user, or management pressures.
- Prototyping allows **flexible adjustments** without redoing the entire system.
- Customers provide feedback on the prototype, and developers refine or modify the system accordingly.
- It prevents expensive rework later in the development process.



Types of Prototyping

1. **Evolutionary Prototyping**
   - **Definition:** The prototype is continuously refined based on user feedback until it evolves into the final system.
   - **Use Case:** Best for projects with **unclear requirements** or **new/complex technologies**.
   - **Example (Banking App):** Start with a login and balance feature. Users request fund transfer and bill payment. Each feature is added iteratively until the app is complete.
2. **Throw-away (Rapid) Prototyping**
   - **Definition:** A prototype is quickly built to clarify requirements, gets user feedback, and is then **discarded**. The final system is developed from scratch using the insights gained.
   - **Use Case:** Useful for exploring **ideas** or **visualizing user interfaces**.
   - **Example (Banking App):** A UI prototype shows account summary and transaction history. Feedback is used to define requirements, but the prototype itself is **not part of the final app**.

2) Incremental Delivery

- **Definition:** An approach where software is developed and delivered in **small, functional increments** rather than a complete system at once.
- **Purpose:** Helps cope with **changing requirements** and lets customers provide feedback on each increment.
- **Process:**
  1. Identify all system services and **prioritize** them.
  2. Define multiple **increments**, each implementing a subset of functionality.
  3. Develop the **first increment** (highest priority features) and deliver it to the customer.
  4. Customer **uses and tests** the increment, providing feedback.
  5. Later increments are **refined and integrated** with previous ones until the full system is complete.
- **Example (Banking App):**
  - **Increment 1:** Account login and balance check.
  - **Increment 2:** Fund transfer and bill payment.
  - **Increment 3:** Loan applications and investment management.
  - Each increment is delivered to the customer for real use and feedback before the next increment is developed.

Process Improvement

Process Improvement involves understanding existing software development processes and modifying them to

increase product quality and/or **reduce cost and development time**.
Approaches:

Aspect	Process Maturity Approach	Agile Approach
What it focuses on	Improving existing processes and project management	Iterative development and reducing unnecessary overheads
Goal	Introduce good software engineering practices and standardize process	Rapid delivery of functionality and quick response to change
How it works	Follows formal stages: planning, documentation, code review, formal testing	Works in small cycles, constantly updates software based on feedback
Flexibility	Less flexible; process improvements are formal and structured	Very flexible; changes can be incorporated quickly
Example	A company introduces strict code review and testing procedures for all projects	A mobile banking app team delivers weekly updates based on user feedback

Software development process models

V-Shape Model

- A software development model where testing activities are planned parallel to each development stage.
- **Characteristics:**
  - Sequential like Waterfall but testing is emphasized at each corresponding development stage.
  - Verification and validation steps are paired with development phases.
  - Not iterative; each phase is completed before moving on.
- **Advantages:**
  - Early test planning reduces defects.
  - Clear milestones make management easy.
  - High guarantee of success if requirements are clear.
- **Disadvantages:**
  - Inflexible to changes; difficult to go back.
  - Costlier than simple Waterfall for minor projects.
- **Example:** Developing a payroll system where each module is verified with its test case before integration.

Spiral Model

- An iterative software development model that combines risk analysis with prototyping and incremental delivery.
- **Characteristics:**
  - Development and testing happen concurrently in cycles called “spirals.”
  - Focuses on risk assessment and reducing project uncertainties.
  - Iterative; the software evolves with each spiral.
- **Advantages:**
  - Reduces project risk through early detection.
  - Flexible to requirement changes.
  - Suitable for large and complex projects.
- **Disadvantages:**
  - Expensive and time-consuming.
  - Requires highly skilled team.
- **Example:** Developing an online banking system where each cycle delivers new features while revisiting previous ones for risk and improvement.

Component-Based Software Engineering (CBSE)

CBSE is a procedure that focuses on designing and developing systems using **existing reusable software components** rather than building from scratch. Also called **reuse-oriented development**.

Process:

- Identify requirements.
- Search for reusable components.
- Integrate components to build the system.
- Develop new components only if needed.

Advantages:

- **Reusability:** Components can be reused in multiple projects.
- **Reduced risk:** Less chance of errors in new development.
- **Faster development:** Quicker than building all components from scratch.
- **Lower cost:** Reduces overall development expenses.
- **Easy to extend:** New functionality can be added by integrating new components.

Example:

- Building an **e-commerce website** by reusing existing payment gateway, authentication modules, and inventory components instead of coding everything from scratch.